



PRINICPLES OF COMPILER DESIGN

(Assignment_03)

NAME: Dagmawi Taye

ID:BDU1505627

Sec:A

Q =#33

Q33. Detect redefinition of variables in the same scope.

You will create a semantic pass that identifies illegal redefinitions within the same scope. The assignment includes handling shadowing correctly in deeper scopes.

```
-----  
  
#include <iostream>  
#include <string>  
#include <vector>  
#include <unordered_map>  
#include <memory>  
#include <sstream>  
#include <algorithm>  
  
enum class NodeType {  
    PROGRAM,  
    FUNCTION_DECL,  
    VARIABLE_DECL,  
    BLOCK,  
    IDENTIFIER,  
    ASSIGNMENT,  
    BINARY_OP  
};  
  
struct ASTNode {  
    NodeType type;  
    std::string value;  
    std::vector<std::shared_ptr<ASTNode>> children;  
    int line;  
    int column;  
  
    ASTNode(NodeType t, std::string v = "", int l = 0, int c = 0)  
        : type(t), value(v), line(l), column(c) {}  
};  
  
class Symbol {  
public:
```

```

    std::string name;
    std::string type;
    int line;
    int column;
    int scopeLevel;

    Symbol(std::string n, std::string t, int l, int c, int sl)
        : name(n), type(t), line(l), column(c), scopeLevel(sl) {}
};

class Scope {
private:
    std::unordered_map<std::string, std::shared_ptr<Symbol>> symbols;
    Scope* parent;
    int level;

public:
    Scope(Scope* p = nullptr, int l = 0) : parent(p), level(l) {}

    // Add a symbol to the current scope
    bool addSymbol(const std::string& name, const std::string& type, int line, int column) {
        // Check if symbol already exists in current scope
        if (symbols.find(name) != symbols.end()) {
            return false; // Redefinition error
        }

        symbols[name] = std::make_shared<Symbol>(name, type, line, column, level);
        return true;
    }

    // Look up a symbol in current and parent scopes
    std::shared_ptr<Symbol> lookup(const std::string& name) {
        // Check current scope first
        auto it = symbols.find(name);
        if (it != symbols.end()) {
            return it->second;
        }
    }

```

```

        // Check parent scope if exists
        if (parent != nullptr) {
            return parent->lookup(name);
        }

        return nullptr; // Symbol not found
    }

    // Look up only in current scope (for redefinition check)
    std::shared_ptr<Symbol> lookupCurrent(const std::string& name) {
        auto it = symbols.find(name);
        if (it != symbols.end()) {
            return it->second;
        }
        return nullptr;
    }

    int getLevel() const { return level; }
    Scope* getParent() const { return parent; }

    // Get all symbols in current scope (for debugging)
    std::vector<std::shared_ptr<Symbol>> getAllSymbols() const {
        std::vector<std::shared_ptr<Symbol>> result;
        for (const auto& pair : symbols) {
            result.push_back(pair.second);
        }
        return result;
    }
};

class SymbolTable {
private:
    Scope* currentScope;
    int scopeCounter;
    std::vector<std::unique_ptr<Scope>> scopes;

public:
    SymbolTable() : currentScope(nullptr), scopeCounter(0) {
        // Create global scope

```

```

    enterScope();
}

~SymbolTable() {
    // Scopes are automatically deleted by unique_ptr
}

void enterScope() {
    currentScope = new Scope(currentScope, scopeCounter++);
    scopes.emplace_back(currentScope);
}

void exitScope() {
    if (currentScope != nullptr) {
        currentScope = currentScope->getParent();
    }
}

Scope* getCurrentScope() const { return currentScope; }

// Add a symbol to current scope
bool addSymbol(const std::string& name, const std::string& type,
               int line, int column) {
    if (currentScope == nullptr) return false;
    return currentScope->addSymbol(name, type, line, column);
}

// Look up a symbol
std::shared_ptr<Symbol> lookup(const std::string& name) {
    if (currentScope == nullptr) return nullptr;
    return currentScope->lookup(name);
}

// Check if a symbol is redefined in current scope
bool isRedefinedInCurrentScope(const std::string& name) {
    if (currentScope == nullptr) return false;
    return currentScope->lookupCurrent(name) != nullptr;
}

```

```

// Get current scope level
int getCurrentScopeLevel() const {
    return currentScope ? currentScope->getLevel() : -1;
}

// Print symbol table for debugging
void printSymbolTable() const {
    std::cout << "\n=== Symbol Table Dump ===\n";
    for (size_t i = 0; i < scopes.size(); ++i) {
        std::cout << "Scope Level " << scopes[i]->getLevel() << ":\n";
        auto symbols = scopes[i]->getAllSymbols();
        for (const auto& sym : symbols) {
            std::cout << "  " << sym->name << " : " << sym->type
                        << " (line " << sym->line << ", col " << sym->column
                        << ", scope " << sym->scopeLevel << ")\n";
        }
        std::cout << std::endl;
    }
}

};

// ===== Semantic Analyzer =====
class SemanticAnalyzer {
private:
    SymbolTable symbolTable;
    std::vector<std::string> errors;
    std::vector<std::string> warnings;

    void addError(const std::string& message, int line, int column) {
        std::ostringstream oss;
        oss << "Error at line " << line << ", column " << column
            << ": " << message;
        errors.push_back(oss.str());
    }

    void addWarning(const std::string& message, int line, int column) {
        std::ostringstream oss;
        oss << "Warning at line " << line << ", column " << column
            << ": " << message;
    }
}

```

```

        warnings.push_back(oss.str());
    }

public:
    // Analyze the AST
    void analyze(const std::shared_ptr<ASTNode>& node) {
        if (!node) return;

        switch (node->type) {
            case NodeType::PROGRAM:
                analyzeProgram(node);
                break;
            case NodeType::FUNCTION_DECL:
                analyzeFunctionDecl(node);
                break;
            case NodeType::VARIABLE_DECL:
                analyzeVariableDecl(node);
                break;
            case NodeType::BLOCK:
                analyzeBlock(node);
                break;
            case NodeType::IDENTIFIER:
                analyzeIdentifier(node);
                break;
            case NodeType::ASSIGNMENT:
                analyzeAssignment(node);
                break;
            default:
                // Analyze children for other node types
                for (const auto& child : node->children) {
                    analyze(child);
                }
                break;
        }
    }

    void analyzeProgram(const std::shared_ptr<ASTNode>& node) {
        // Global scope is already entered in SymbolTable constructor
        for (const auto& child : node->children) {

```

```

        analyze(child);
    }
}

void analyzeFunctionDecl(const std::shared_ptr<ASTNode>& node) {
    // Add function name to current scope
    std::string funcName = node->value;

    if (symbolTable.isRedefinedInCurrentScope(funcName)) {
        addError("Redefinition of function '" + funcName + "'",
            node->line, node->column);
    } else {
        symbolTable.addSymbol(funcName, "function", node->line, node->column);
    }

    // Enter function scope
    symbolTable.enterScope();

    // Analyze parameters (simplified - assume first child is parameters)
    if (!node->children.empty()) {
        // In a real compiler, you would analyze parameters here
    }

    // Analyze function body
    for (const auto& child : node->children) {
        analyze(child);
    }

    // Exit function scope
    symbolTable.exitScope();
}

void analyzeVariableDecl(const std::shared_ptr<ASTNode>& node) {
    if (node->children.empty()) return;

    // Assuming first child is identifier, second is type (simplified)
    auto identifierNode = node->children[0];
    std::string varName = identifierNode->value;

```



```

// Check for redefinition in current scope
if (symbolTable.isRedefinedInCurrentScope(varName)) {
    addError("Redefinition of variable '" + varName + "' in the same scope",
            identifierNode->line, identifierNode->column);
} else {
    // Determine variable type (simplified)
    std::string varType = "unknown";
    if (node->children.size() > 1) {
        varType = node->children[1]->value;
    }

    // Add to symbol table
    symbolTable.addSymbol(varName, varType,
            identifierNode->line, identifierNode->column);
}

// Analyze initializer if exists
for (size_t i = 2; i < node->children.size(); ++i) {
    analyze(node->children[i]);
}
}

void analyzeBlock(const std::shared_ptr<ASTNode>& node) {
    // Enter new scope for block
    symbolTable.enterScope();

    // Analyze statements in block
    for (const auto& child : node->children) {
        analyze(child);
    }

    // Exit block scope
    symbolTable.exitScope();
}

void analyzeIdentifier(const std::shared_ptr<ASTNode>& node) {
    std::string name = node->value;
    auto symbol = symbolTable.lookup(name);
}

```

```

        if (!symbol) {
            addError("Undeclared identifier '" + name + "'",
                    node->line, node->column);
        }
    }

}

void analyzeAssignment(const std::shared_ptr<ASTNode>& node) {
    // Check if left side is a declared variable
    if (!node->children.empty()) {
        auto leftNode = node->children[0];
        if (leftNode->type == NodeType::IDENTIFIER) {
            analyzeIdentifier(leftNode);
        }
    }

    // Analyze right side
    for (size_t i = 1; i < node->children.size(); ++i) {
        analyze(node->children[i]);
    }
}

// Get analysis results
void printResults() const {
    std::cout << "\n=== Semantic Analysis Results ===\n";

    if (errors.empty() && warnings.empty()) {
        std::cout << "No semantic errors or warnings found.\n";
    }

    if (!warnings.empty()) {
        std::cout << "\nWarnings:\n";
        for (const auto& warning : warnings) {
            std::cout << "    " << warning << "\n";
        }
    }

    if (!errors.empty()) {
        std::cout << "\nErrors:\n";
    }
}

```

```

        for (const auto& error : errors) {
            std::cout << "    " << error << "\n";
        }
        std::cout << "\nCompilation failed due to semantic errors.\n";
    } else if (!warnings.empty()) {
        std::cout << "\nCompilation succeeded with warnings.\n";
    } else {
        std::cout << "\nCompilation succeeded.\n";
    }
}

const std::vector<std::string>& getErrors() const { return errors; }
const std::vector<std::string>& getWarnings() const { return warnings; }

void printSymbolTable() {
    symbolTable.printSymbolTable();
}

};

// ===== Test Cases =====
std::shared_ptr<ASTNode> createTestAST_Valid() {
    // Test case 1: Valid program with shadowing
    // {
    //     int x = 10;
    //     {
    //         int x = 20; // Valid shadowing
    //         int y = 30;
    //     }
    //     int y = 40; // Valid - different scope
    // }

    auto program = std::make_shared<ASTNode>(NodeType::PROGRAM, "program");

    // Outer block
    auto outerBlock = std::make_shared<ASTNode>(NodeType::BLOCK, "", 1, 1);

    // int x = 10
    auto varDecl1 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 2, 3);
    auto id1 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "x", 2, 7);

```

```

auto type1 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 2, 3);
varDecl1->children.push_back(id1);
varDecl1->children.push_back(type1);

// Inner block
auto innerBlock = std::make_shared<ASTNode>(NodeType::BLOCK, "", 4, 3);

// int x = 20 (shadowing)
auto varDecl2 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 5, 5);
auto id2 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "x", 5, 9);
auto type2 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 5, 5);
varDecl2->children.push_back(id2);
varDecl2->children.push_back(type2);

// int y = 30
auto varDecl3 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 6, 5);
auto id3 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "y", 6, 9);
auto type3 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 6, 5);
varDecl3->children.push_back(id3);
varDecl3->children.push_back(type3);

innerBlock->children.push_back(varDecl2);
innerBlock->children.push_back(varDecl3);

// int y = 40 (in outer scope)
auto varDecl4 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 8, 3);
auto id4 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "y", 8, 7);
auto type4 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 8, 3);
varDecl4->children.push_back(id4);
varDecl4->children.push_back(type4);

outerBlock->children.push_back(varDecl1);
outerBlock->children.push_back(innerBlock);
outerBlock->children.push_back(varDecl4);

program->children.push_back(outerBlock);
return program;
}

```

```

std::shared_ptr<ASTNode> createTestAST_Redefinition() {
    // Test case 2: Invalid redefinition in same scope
    // {
    //     int x = 10;
    //     int x = 20; // ERROR: Redefinition in same scope
    //     {
    //         int x = 30; // Valid shadowing
    //     }
    // }

    auto program = std::make_shared<ASTNode>(NodeType::PROGRAM, "program");
    auto block = std::make_shared<ASTNode>(NodeType::BLOCK, "", 1, 1);

    // int x = 10
    auto varDecl1 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 2, 3);
    auto id1 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "x", 2, 7);
    auto type1 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 2, 3);
    varDecl1->children.push_back(id1);
    varDecl1->children.push_back(type1);

    // int x = 20 (ERROR: redefinition)
    auto varDecl2 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 3, 3);
    auto id2 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "x", 3, 7);
    auto type2 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 3, 3);
    varDecl2->children.push_back(id2);
    varDecl2->children.push_back(type2);

    // Inner block with valid shadowing
    auto innerBlock = std::make_shared<ASTNode>(NodeType::BLOCK, "", 4, 3);
    auto varDecl3 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 5, 5);
    auto id3 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "x", 5, 9);
    auto type3 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 5, 5);
    varDecl3->children.push_back(id3);
    varDecl3->children.push_back(type3);
    innerBlock->children.push_back(varDecl3);

    block->children.push_back(varDecl1);
    block->children.push_back(varDecl2);
    block->children.push_back(innerBlock);
}

```

```

    program->children.push_back(block);

    return program;
}

std::shared_ptr<ASTNode> createTestAST_Function() {
    // Test case 3: Function with parameters and local variables
    // function foo(int a, int b) {
    //     int result = a + b;
    //     {
    //         int temp = result * 2; // Valid shadowing not happening
    //     }
    //     return result;
    // }

    auto program = std::make_shared<ASTNode>(NodeType::PROGRAM, "program");
    auto funcDecl = std::make_shared<ASTNode>(NodeType::FUNCTION_DECL, "foo", 1, 1);

    // Function body block
    auto funcBody = std::make_shared<ASTNode>(NodeType::BLOCK, "", 1, 20);

    // int result = a + b
    auto varDecl1 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 2, 3);
    auto id1 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "result", 2, 7);
    auto type1 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 2, 3);
    varDecl1->children.push_back(id1);
    varDecl1->children.push_back(type1);

    // Inner block
    auto innerBlock = std::make_shared<ASTNode>(NodeType::BLOCK, "", 3, 3);

    // int temp = result * 2
    auto varDecl2 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 4, 5);
    auto id2 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "temp", 4, 9);
    auto type2 = std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 4, 5);
    varDecl2->children.push_back(id2);
    varDecl2->children.push_back(type2);

    innerBlock->children.push_back(varDecl2);

```

```

funcBody->children.push_back(varDecl1);
funcBody->children.push_back(innerBlock);
funcDecl->children.push_back(funcBody);
program->children.push_back(funcDecl);

return program;
}

std::shared_ptr<ASTNode> createTestAST_Complex() {
    // Test case 4: Complex nested scopes with mixed declarations
    // {
    //   int a = 1;
    //   {
    //     int b = 2;
    //     int a = 3; // Valid shadowing
    //     {
    //       int c = 4;
    //       int b = 5; // Valid shadowing
    //     }
    //     int c = 6; // Valid - previous c is in different scope
    //   }
    //   int b = 7; // Valid - previous b is in different scope
    //   int a = 8; // ERROR: Redefinition in same scope
    // }

    auto program = std::make_shared<ASTNode>(NodeType::PROGRAM, "program");
    auto block1 = std::make_shared<ASTNode>(NodeType::BLOCK, "", 1, 1);

    // Level 1: int a = 1
    auto var1 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 2, 3);
    var1->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "a", 2,
7));
    var1->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 2
, 3));

    // Level 2 block
    auto block2 = std::make_shared<ASTNode>(NodeType::BLOCK, "", 3, 3);

    // Level 2: int b = 2

```

```

    auto var2 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 4, 5);
    var2->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "b", 4,
9));
    var2->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 4
, 5));

    // Level 2: int a = 3 (shadowing)
    auto var3 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 5, 5);
    var3->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "a", 5,
9));
    var3->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 5
, 5));

    // Level 3 block
    auto block3 = std::make_shared<ASTNode>(NodeType::BLOCK, "", 6, 5);

    // Level 3: int c = 4
    auto var4 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 7, 7);
    var4->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "c", 7,
11));
    var4->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 7
, 7));

    // Level 3: int b = 5 (shadowing)
    auto var5 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 8, 7);
    var5->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "b", 8,
11));
    var5->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 8
, 7));

    block3->children.push_back(var4);
    block3->children.push_back(var5);

    // Back to Level 2: int c = 6
    auto var6 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 10, 5);
    var6->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "c", 10,
9));
    var6->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 1
0, 5));

```



```

    block2->children.push_back(var2);
    block2->children.push_back(var3);
    block2->children.push_back(block3);
    block2->children.push_back(var6);

    // Back to Level 1: int b = 7
    auto var7 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 12, 3);
    var7->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "b", 12,
7));
    var7->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 1
2, 3));

    // Back to Level 1: int a = 8 (ERROR: redefinition)
    auto var8 = std::make_shared<ASTNode>(NodeType::VARIABLE_DECL, "", 13, 3);
    var8->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "a", 13,
7));
    var8->children.push_back(std::make_shared<ASTNode>(NodeType::IDENTIFIER, "int", 1
3, 3));

    block1->children.push_back(var1);
    block1->children.push_back(block2);
    block1->children.push_back(var7);
    block1->children.push_back(var8);
    program->children.push_back(block1);

    return program;
}

// ===== Main Test Function =====
void runTest(const std::string& testName, std::shared_ptr<ASTNode> ast) {
    std::cout << "\n" << std::string(60, '=') << "\n";
    std::cout << "Running Test: " << testName << "\n";
    std::cout << std::string(60, '=') << "\n";

    SemanticAnalyzer analyzer;
    analyzer.analyze(ast);
    analyzer.printResults();
    analyzer.printSymbolTable();
}

```

```

}

int main() {
    std::cout << "Semantic Analyzer - Variable Redefinition Detection\n";
    std::cout << "Handles scope management and detects illegal redefinitions\n\n";

    // Run all test cases
    runTest("Valid Program with Shadowing", createTestAST_Valid());
    runTest("Redefinition in Same Scope", createTestAST_Redefinition());
    runTest("Function with Local Variables", createTestAST_Function());
    runTest("Complex Nested Scopes", createTestAST_Complex());

    std::cout << "\n" << std::string(60, '=') << "\n";
    std::cout << "All tests completed.\n";

    return 0;
}

```

Key Features of this Implementation:

1. Symbol Table with Scope Management

- **Scope class:** Manages symbols at each scope level with parent-child relationships
- **SymbolTable class:** Maintains a stack of scopes with enter/exit operations
- **Proper shadowing:** Variables can be redefined in deeper scopes (shadowing)
- **Redefinition detection:** Flags errors when variables are redefined in the same scope

2. Semantic Analysis Rules

- **Same scope redefinition:** Detected as an error
- **Different scope redefinition:** Allowed (shadowing)
- **Undeclared variable usage:** Detected as an error
- **Scope hierarchy:** Maintains proper nesting of scopes

3. AST Representation

- Simplified AST node structure for demonstration
- Supports blocks, variable declarations, functions, and identifiers
- Line and column information for accurate error reporting

4. Test Cases Included

- **Valid shadowing:** Variables redefined in inner scopes
- **Invalid redefinition:** Same variable in same scope
- **Function scope:** Variables in function bodies
- **Complex nesting:** Multiple levels of scope nesting

5. Error Reporting

- Clear error messages with line and column numbers
- Symbol table dumping for debugging
- Separation of errors and warnings

How to Use and Extend:

1. Compile and Run:

```
bash
g++ -std=c++11 semantic_analyzer.cpp -o semantic_analyzer
./semantic_analyzer
```

2. Extend for Real Language:

- Add type checking logic in analyzeVariableDecl.
- Implement function parameter analysis
- Add support for more AST node types
- Integrate with your parser's AST generation

3. Integration with Full Compiler:

- Connect this semantic analyzer after parsing phase
- Use the AST from your parser
- Store semantic information for code generation phase

This implementation correctly handles the assignment requirement to "identify illegal redefinitions within the same scope while handling shadowing correctly in deeper scopes." The modular design allows easy extension for full semantic analysis including type checking, function overloading, and other semantic rules.